

GUMP – A Minimal GIMP Application

IG. 2409 – Multimedia Applications

Asa Levine

1. Course Overview

During the duration of IG.2409: Multimedia Applications, both the principles of Object Oriented Programming (OOP) in C++ and the techniques for image manipulation with OpenCV¹ have been taught to facilitate the creation of applications handling image editing on our own. It began with a simple overview of the C++ language and its I/O interface. However, that would quickly evolve into a deeper dive into the OpenCV library – one of the biggest open source computer vision libraries for both C++ and Python.

The course first introduced file reading and writing alongside simple operations like brightness control to get acclimated to the new library. This would continue further to both image and video manipulation with contrast, blur, mouse events, and more. The work would deeply familiarize one with the basic tools available to a user of the OpenCV library. This all culminates in the final project: a minimal GIMP application. This would combine different aspects of earlier coursework into a custom application to allow a user to modify images (and videos if that path was pursued) according to a number of different methods – though the specifics depend on the implementation. This is one of such projects: GUMP.

¹ The OpenCV library contains both versions cv and cv2. The latter is used throughout this project, as the older cv version contains outdated c-api bindings while offering no general upside against cv2.

2. Approach

All functionality for the GUMP application² is divided up according to methodologies pulled from Object Oriented Programming (OOP) – divided up into four primary subdivisions: the main application, manipulation methods, a user input handler, and a `Record` class for storing modification history. The main application file itself houses only a file loader, the exit handler, and the main framework logic itself. The division of these was decided upon simply by their functionalities and what they interact with. The divisions – following OOP guidance – keeps the code clean and legible while reducing redundancy in code blocks. These are then all linked together using a CMake build process for an efficient and lightweight application.

Regarding the aforementioned manipulation methods, another subdivision can be made between the methods that directly manipulate the image and the ones that manipulate the user's workflow. For the former, there are a variety of methods accessible to the user, include all of the following:

- *Dilate*
- *Erode*
- *Resize*

² All source code can be found at the GitHub repository linked below:
<https://github.com/arcreane/gimpsep-gump>

- *Flip*
- *Lighten*
- *Darken*
- *Stitch*
- *Canny Edge Detection*

Additionally, for the workflow manipulation methods, the following are included accessible to the user:

- *Restore*
- *Save*
- *Undo*
- *Redo*
- *Exit*
- *History*

A brief example of what a sequences of some of the above inputs may look like during image modification is as follows:

```

Enter the name of the file you'd like to load:
davec.jpg
Please enter your desired operation:
Editing : [Dilate, Erode, Resize, Flip, Lighten, Darken, Stitch, Canny (for Canny Edge Detection)]
File    : [History, Restore, Undo, Redo, Save, Exit]

flip
would you like flip horizontally or vertically:
Vertically
Please enter your desired operation:
Editing : [Dilate, Erode, Resize, Flip, Lighten, Darken, Stitch, Canny (for Canny Edge Detection)]
File    : [History, Restore, Undo, Redo, Save, Exit]

history
Current Recorded Operations:
| FLIP <<< (current)
| ORIGINAL

Please enter your desired operation:
Editing : [Dilate, Erode, Resize, Flip, Lighten, Darken, Stitch, Canny (for Canny Edge Detection)]
File    : [History, Restore, Undo, Redo, Save, Exit]

save
Please enter the name of the output file (extension included):
davecFlipped.jpg
would you like to continue editing operations [y/n]:
y
Please enter your desired operation:
Editing : [Dilate, Erode, Resize, Flip, Lighten, Darken, Stitch, Canny (for Canny Edge Detection)]
File    : [History, Restore, Undo, Redo, Save, Exit]

exit

```

Fig 1. An example sequence of inputs to GUMP opening, flipping, and saving the image before exiting.

Image Transformation. This section will briefly walk through each of the available image transformation methods and give some detail on the development process. All of these methods directly influence the image(s) uploaded in their unique ways.

Dilate/Erode. The `dilate` and `erode` are two similar functions that operate in the same, but opposite, ways. Dilation adds pixels to the boundaries of objects within an image, while erosion removes those pixels. The user input needed is to define how many iterations the transformation is applied as well as the kernel size that applies the transformation. Then, by combining OpenCV's `getStructuringElement` and `dilate` or `erode` respectively, the method applies the necessary transformation to the image.

Resize. The `resize` method, as its namesake suggests, resizes the passed image's dimensions by a scale factor. This is accomplished by prompting the user for a scale factor to modify the image by, then using OpenCV's `resize` method with either the `INTER_LINEAR` (for upsizing) or `INTER_AREA` (for downsizing) interpolation method depending on the new size.

Flip. `flip` is the simplest of the methods; it simply asks the user whether the desired flip is vertical or horizontal. It then executes that flip using OpenCV's `flip`, passing in the matching value to the user's desired change.

Lighten/Darken. `lighten` and `darken` are two of the simpler methods, and two sides to the same coin (much like the prior `dilate` and `erode`). They both operate by simply taking the most recent image on record and asking the user for a level of lightening or darkening respectively. It then applies that value using OpenCV's `convertTo` method. This method applies the beta value across the board to all pixels

within the image, lightening or darkening respectively.

Stitch. The `stitch` method is by and far the most involved of all the transformations. This is due to the fact that it doesn't involve just one image, but multiple, and how it handles those images depends on the type of stitching it's doing (Scan vs. Panorama). The algorithm implemented for GUMP was one that allows for inclusion of the originally loaded image, and is the first check of the process. It then continues by prompting the user for all other files they wish to load. After parsing all these, it checks for the mode, either Scan or Panorama. It then creates a `stitcher` object from OpenCV's `Stitcher` class (as well as a status tracker from that same class). The program then attempts to stitch the photos provided together, setting the status to OK on success, or one of three other failure statuses should the stitching have been unsuccessful.

Canny Edge Detection. Canny edge detection – named after the developer John F. Canny in 1986 – is an algorithm designed to highlight edges in an image while removing the rest.³ This algorithm is considered one of the foremost edge detection algorithms, both for its low error rate and high efficiency. OpenCV has a built-in method in the form of `Canny`. This method takes in two thresholds and a kernel size for applying the edge detection algorithm. These two thresholds, low and

high, set the definitions for what counts as edges. Below the low threshold is noise. Between the two is a weak edge. Above the high threshold is considered a strong edge. With this, the user has control over how strong they wish for the edge detection algorithm to be upon the image. After computation, the method returns a new grayscale image with clearly defined edges given the strength and finesse determined by thresholds and the kernel size respectively.

Image Workflow. Similar to the last section, this part will go over all of the image workflow methods available to the user and their underlying architecture as well. The workflow describes not the actual editing transformations done to the image, but everything else the user can do to assist in their editing process. This involves program commands as well as image version control and history.

Restore. The `restore` method is fairly simple: it wipes all editing progress from the image – bringing it back to the original loaded image. It is irreversible and is designed in order to be used only in the case of a full clear being needed deep into an editing sequence. It does this by simply accessing the source value stored within the record, wiping the record, and pushing that source back into it. Thus, it starts the user back at square one again.

Save. The `save` method functions fairly simply – prompting the user for a file name and writing the file to the system. It then asks if the user wishes to continue the process: if no it exists and if yes it saves and

³ The algorithm is a fairly involved process, and the full details on the inner workings and mechanisms can be found here:
<https://www.educative.io/answers/what-is-canny-edge-detection>

reverts to the initial menu. All that's used from the OpenCV library is the `imwrite` method, while the rest is handled by the custom user input handler.

Undo/Redo. The Undo and Redo functions available to the user are where the `Record` class comes into play properly. Undo rewinds the last operation (if it exists), and Redo shifts to a more recent operation (if it exists). This is possible due to the record of alterations kept by GUMP during runtime. The length of this record is variable, but at default is set to one hundred entries.

Exit. `exit` is a fairly straightforward method: it attempts to quit the program. If the latest image is either the original or it was just saved, the program simply quits. However, if the image has unsaved edits, the method prompts the user if they would like to cancel the operation, save and quit, or quit without saving. The first option simply resumes normal operation. The second goes through the saving process described above. The third simply immediately exits the program, discarding all unsaved changes.

History. History is one of the simpler actions, as it simply prints out the most recent ten operations, with an arrow to the most current. It's simply for the user to have a better visual with what has happened, especially when they're redoing and undoing multiple times over after long strings of edits. This is done by referencing the `Record` object that handles all image version control and history tracking.

3. Challenges

A number of challenges arose during the creation of GUMP, both in terms of the code functionality and user experience. For a majority of these, the access to a number of the tools discussed in the next section combined with some code restructuring would do away with the problem. However, due to the constraints (both in time and complexity), a few problems lacked an optimal solution – most notably the ____ (more on those more difficult problems at the end of this section).

Minor Challenges. One of the first minor challenges encountered was simply setting up the linking stages itself. A number of library installation issues reared their proverbial heads at the beginning, and even after that was sorted out, issues during the build process existed for a short time. However, a short bit of research would yield all the necessary information and installations needed to get the library running. From there, there were a few issues regarding the standard input over-consuming user inputs during failed validation stages as well as some version control issues with the record system, but all of these were easily resolved with a little bit of time and research into the desired functionality. The later mentioned tools section includes these resources more specifically.

Record Design Challenges. One of the biggest challenges faced was designing an efficient record system. From the start, a record system felt to be a needed inclusion, as being able to undo mistaken or unwanted edits (and restore should the rewind be the

unwanted one), but exactly how this was developed underwent a few changes. The first design was simple: a list of pointers directed at `Mat` objects. It was straightforward and provided a somewhat solid framework, but lacked much of the framework and ease of use that was required and desired for the application.

It would then undergo multiple iterations, growing in complexity each time. First, the list was swapped from pointers to the actual `Mat` objects. This was done both to simplify some code within the `Record` class, but also to remove some now redundant code within the main file. Moreover, a few supporting attributes like a max size and an index for the current image were added to facilitate new methods. A custom `push` operation was added to maintain the queue and `curIndex` values, as well as methods for fetching the current image, prior image (if it exists), and future image (if it exists). These latter two are also how the undo and redo functions work. These would begin to actually form the final design of the `Record` class. However, new issues popped up at this point. First, how does one detect if a prior or future iteration exists? Moreover, how does a complete restoration interact with this?

These questions would lead to the introduction of two new attributes: a source variable and an operation record. The source variable is the simpler of the two – it simply stores the original image `Mat`, and in the case of restoration, it clears the record and sets the current to the source. Ideally, if a little more time was had, the restoration would be reversible. However, this adds a

fair bit of complexity to the record that wasn't able to be resolved well in the given time. And while that complexity is mentioned – much of the complexity in the `Record` class comes from the iteration tracking. As functions, undo and redo need to be able to do a few different things. First: they must preserve the past, present, and future. Second: they must skip operations that don't alter the image (all of the workflow operations). The first of this is simple enough; keeping a queue with iterations stored solves that. However, the second becomes more complicated. The first thought was to just not include those changes, and that works alright for the most part. However, this is where the `exit` method rears its head. This method checks if there exists any unsaved changes before quitting, which requires the record to now store if the image is the original and when it has been saved. This is why the operation record exists. This string queue stores the operation string alongside the record queue as well, and it allows both the `exit` method to check for that while also keeping the undo and redo functions able to move between versions, as it can now skip over save operations.

Now combine all the development above with a few other method like `getters/setters` for the source, `clear`, `toString`, and a method for fetching the most recent string operation in `getMostRecentOperation`, and the `Record` class is complete (for the purposes of this project at least). The iterations made have improved the class, both in usability

and efficiency significantly – making it the core piece of the entire application.

Future Integrations. When mentioning some of the major challenges before, it was mentioned not all had an optimal solution. Some of these ended up relegated to “what ifs” due to time constraints, but theoretically, if the time existed, a number of additions would be made. First, a proper graphical user interface (GUI) would be integral in improving user experience. However, the creation and design of a good GUI is a time consuming process that simply was outside of the scope of this project to the degree that was desired. Second would be the introduction of transformation like crops, selections, and cuts that are cursor-based, taking the manipulation outside of the console. This requires the aforementioned GUI to function well (as technically it is possible without it, but it was found lacking in cleanliness), and thus this functionality was also pushed into this theoretical future bucket should further time have existed.⁴

4. Tools

A small handful of tools provided instrumental in the development of GUMP. Of course, the first of these is the OpenCV documentation itself. From the actual methods’ notation and information to examples on some of the more complex methods, it was instrumental in properly using the library’s breadth of options. When using any new library, having these

resources are extraordinarily helpful in getting a grip on its functionality. Especially with cases like the `stitch` method, having the given examples⁵ made the entire process far simpler.

Additionally, online forums like StackOverflow and the OpenCV forums were instrumental in some of the more specific issues and implementation requirements. While the documentation is immensely helpful, it has limits. It provides the information needed to understand what it does and what each part is. However, the more specific use cases and niche interactions a library’s contents can have with both one another and other aspects of the code can be more difficult to glean from just the code documentation. It’s in these cases where the aforementioned forums come into play. By looking through others’ challenges and their solutions, it can give a ton of insight into what may be going wrong. For instance, one issue that was run into during the development process was with the `getStructuringElement` method for erosion and dilation. While the method returns a `Mat`, it immediately passes it into `erode/dilate` as an `InputArray`. However, if the returned structuring element is implicitly cast to an `InputArray`, the `Mat`’s dimensions are cleared, resulting in a dimensionality error being thrown. Issues like these can be tougher to debug with just documentation, but by consulting the experiences of others and seeing similar

⁴ One may note the lack of a ‘team’ section in this paper. This is due to the fact this was a solo effort as a result of a serious injury sustained during the semester. Further discussion of this can be found by direct contact as alevine.cpslo@gmail.

⁵ This example can be found in the stitcher docs in the OpenCV documentation here: https://docs.opencv.org/4.x/d8/d19/tutorial_stitcher.html

issues, it turned a more difficult bug into a notability more trivial one.

5. Conclusion

First, one must look at where GUMP can be most directly improved. This was discussed shortly prior, but a more expansive look was felt warranted due to the complexity allowed by OpenCV and the prevalence computer vision has in today's technological space. While the introduction of a GUI and mouse-based transformation functionality would be the immediate following steps should more time have been available, they by no means would've been the last. The next steps would first involve some improvements to the `Record` class to allow for more fine-grained control and more data to be stored for better version control. After that, the next step would be to add significantly more diversity in accepted file transformations. This one is simple in concept but a deceptive amount of development time, as each significantly new file format requires rewriting of all the methods to fit that. Moreover, it requires reprogramming the input sequences to be modular against any file format. That said, it's an important part to a truly robust editor, and would be a key feature should development continue properly. The entire project has driven home what is important to think about in application development – especially regarding what's worth the effort against the cost behind the scenes.

More than that, it's important to reflect on what the course's impact is now that its conclusion is quickly approaching. IG. 2409 has given ample time to become familiar

with both a prominent computer vision module in OpenCV and in the process one goes through to use big libraries in a small application. These skills are paramount in a world where tech has become as essential as water in many places, and the development of such software increasingly relies on larger libraries, plugins, and API's. To be well versed in these is to be well prepared for a future of development. That said, application development skills uniquely branch out far wider than simple software development. To have a solid base in application development gives one the ability to integrate technical aspects where they would otherwise be lacking. From wildlife research to business management to personal projects, having the knowhow and resources to be able to produce one's own application can be huge for the efficacy of whatever it's applied to. In this way, what was learned is one of the most important skills to learn for anyone, software developer or otherwise. Applications are used nearly universally, and thus understanding grants capability.